

Асинхронный клиент Redis (или KeyDB)

Асинхронный доступ к Redis

Несмотря на высокую производительность операций над данными, Redis-сервер является сетевым ресурсом, и любое обращение к нему подвержено сетевым задержкам. Для пользователя Redis задержки обращения к серверу означают простой вычислителя и увеличение длительности обработки запросов. Если в случае жёсткой последовательной логики вызывающей стороны, сетевые задержки – это необходимое зло, параллелизуемые процедуры могут выполнять несвязанные инструкции во время ожидания отклика сетевого ресурса.

Платформа предоставляет механизм асинхронного доступа к серверу Redis в виде асинхронного клиента [AsyncClient](#).

Подключение к Redis

Клиенты Redis для ускорения повторных вызовов используют пул соединений с сервером, работающий по "ленивой" схеме, при которой реальное подключение к серверу производится при выполнении первой команды по соединению.

По умолчанию при подключении для аутентификации на кластере Redis клиент использует пользовательскую роль `default` из описания кластера, если же кластер не защищён подключение производится без аутентификации. При необходимости можно указать другую пользовательскую роль для аутентификации на кластере.

Подключение к кластеру Redis

Типичный сценарий - подключение по имени кластера (названию параметра из конфигурации):

```
1 redis::AsyncClient client{ redis::AsyncClient::FromClusterName,
2                             L"Модуль.Хранилище"_s };
3
4 // С аутенфикацией под пользовательской ролью "admin".
5 redis::AsyncClient client{ redis::AsyncClient::FromClusterName,
6                             L"Модуль.Хранилище"_s, L"admin"_s };
```

Подключение с передачей структуры [ClusterEndpoint](#):

```
1 ClusterEndpoint endpoint
2     = Config::Instance().GetOrThrow< ClusterEndpoint >(
3     L"Модуль.Хранилище"_s );
4
5 redis::AsyncClient client{ endpoint, L"Модуль.Хранилище"_s };
6
7 // С аутенфикацией под пользовательской ролью "admin".
8 redis::AsyncClient client{ endpoint, L"Модуль.Хранилище"_s, L"admin"_s
9                             };;
```

Прямое подключение к узлу кластера Redis

При необходимости можно подключиться к конкретному узлу кластера Redis с использованием структуры [Endpoint](#):

```
1 // По имени кластера (названию параметра из конфигурации).
2 redis::Endpoint endpoint
3     = redis::Endpoint::FromClusterName( L"Модуль.Хранилище"_s,
4     L"slave1"_s );
5
6 redis::AsyncClient client{ endpoint };
7
8 // С аутенфикацией под пользовательской ролью "admin".
```

```

7  redis::Endpoint endpoint
8      = redis::Endpoint::FromClusterName( L"Модуль.Хранилище"_s,
      L"slave1"_s, L"admin"_s );
9  redis::AsyncClient client{ endpoint };
10
11
12  // По точке подключения к кластеру.
13  ClusterEndpoint cluster_endpoint
14      = Config::Instance().GetOrThrow< ClusterEndpoint >(
      L"Модуль.Хранилище"_s );
15  redis::Endpoint endpoint
16      = redis::Endpoint::FromClusterEndpoint( cluster_endpoint,
17                                              L"Модуль.Хранилище"_s,
18                                              L"slave1"_s );
19  redis::AsyncClient client{ endpoint };
20
21  // С аутентификацией под пользовательской ролью "admin".
22  redis::Endpoint endpoint
23      = redis::Endpoint::FromClusterEndpoint( cluster_endpoint,
24                                              L"Модуль.Хранилище"_s,
25                                              L"slave1"_s,
26                                              L"admin"_s );
27  redis::AsyncClient client{ endpoint };

```

Прямое подключение к серверу Redis

Для целей локального тестирования можно воспользоваться подключением к серверу Redis по имени узла или IP-адресу с использованием структуры [Endpoint](#):

```

1  redis::Endpoint endpoint{ .mHost = L"127.0.0.1"_s, .mPort = 4321 };
2  redis::AsyncClient client{ endpoint };
3
4  // С аутентификацией под пользователем "user" с паролем "pass".
5  redis::Endpoint endpoint{ .mHost = L"127.0.0.1"_s, .mPort = 4321,
6                          .mUsername = L"user"_s, .mPassword =
        L"pass"_s };
7  redis::AsyncClient client{ endpoint };

```

Подключение к кластеру Redis с использованием строки подключения

Не рекомендовано!

При использовании строки подключения клиент не получает полных сведений о кластере Redis, к одному из серверов которого подключается, и не может скорректировать логику взаимодействия с сервером.

Как следствие, в некоторых редких сценариях могут возникать ошибки подключения, и даже возможно подключение к серверам в других кластерах.

```

1  redis::AsyncClient client{ redis::AsyncClient::FromConnectionString,
2                          L"host='10.10.10.10' port='1234'
        dbindex='2'"_s };
3
4  // С аутентификацией под пользователем "user" с паролем "pass".
5  redis::AsyncClient client{ redis::AsyncClient::FromConnectionString,
6                          L"host='10.10.10.10' port='1234'
        dbindex='2'"_s,

```

Поля `port` (по умолчанию — 6379) и `dbindex` (по умолчанию — 0) являются необязательными.

Команды Redis

Команды, поддерживаемые асинхронным клиентом, реализованы отдельными классами в пространстве имён `redis::commands`. Названия команд и их аргументов соответствуют официальным именам из документации Redis. Описание команд можно найти в документации Redis: [Command reference – Redis](#).

Аргументы команд

Аргументы команд задаются при конструировании:

```
redis::commands::Set command{ L"ключ"_sv, L"значение"_sv };
```

И могут передаваться одним из трёх способов:

1. По lvalue-ссылке (именованной переменной) на владеющий объект (String, ByteVector):

```
1 String value = GetValue();
2 redis::commands::Set command{ L"ключ"_sv, value };
```

2. По rvalue-ссылке (временным объектом) на владеющий объект (String, ByteVector):

```
1 String GetKey();
2 redis::commands::Set command{ GetKey(), L"значение"_sv };
```

3. Невладеющим представлением ([StringView](#), [ArrayView<Byte>](#)):

```
1 String value = GetValue();
2 redis::commands::Set command{ L"ключ"_sv, StringView{ value } };
```

Параметры конструкторов команд реализованы таким образом, чтобы устранить лишние копирования аргументов. Поэтому при выборе способа инициализации следует руководствоваться областью видимости аргументов. При передаче по lvalue-ссылке или с помощью представления область видимости владеющего объекта должна пережить исполнение команды, т.к. в этом случае будет захвачена только ссылка на объект. В то время как переданный по rvalue-ссылке объект захватывается по значению и перемещается в хранилище команды, время жизни которого совпадает со временем жизни команды.

Аргументы-контейнеры

Некоторые команды в качестве аргументов принимают контейнеры ключей, значений или пар "ключ-значение":

```
1 std::map< String, ByteVector > field_values = GetFieldValues();
2 redis::commands::HMSet command{ L"ключ"_sv, field_values };
```

Способы передачи и принцип их выбора аналогичен одиночным аргументам.

Сложные аргументы

В некоторых сценариях может возникать ситуация, когда в качестве аргумента команды требуется передать поле/свойство другого объекта. При этом желательно избежать копирования данных, особенно если аргументом является контейнер.

Для этого параметры конструкторов команд принимают функторы, трансформирующие исходный объект или его элементы в представление аргумента команды ([StringView](#), [ArrayView<Byte>](#)):

```
1 std::map< String, ByteVector > field_values = GetFieldValues();
2 redis::commands::MGet command{
3     { field_values,
```

```

4 |         []( const auto& field_value ) { return StringView{
    |         field_value.first }; } } };

```

Варианты передачи исходного объекта и принцип их выбора такие же.

Возвращаемые значения

Таблица трансляции типов данных Redis в C++ типы SBIS:

- Простые строки (статусы успеха) → void.
- Ошибки (статусы неудачи) → [RedisException](#).
- Целые числа → Int64.
- Целые числа, ограниченные значениями 0 и 1 → bool.
- Объемные строки → String, Binary, OptString, OptBinary.
- Массивы → StringList, BinaryList, OptStringList, OptBinaryList.
- Массивы уникальных значений → StringSet, BinarySet, OptStringSet, OptBinarySet.
- Массивы пар ключ-значение → StringDict, BinaryDict, OptStringDict, OptBinaryDict.

Классы команд, результат которых – объёмная строка или массив, поддерживают выбор возвращаемого значения путём явного задания шаблонного аргумента:

```

1 | // По умолчанию возвращается StringList.
2 | redis::commands::LRange command{ L"ключ"_sv, 1, 7 };
3 | // Выбираем OptBinaryList в качестве результата.
4 | redis::commands::LRange< OptBinaryList > command{ L"ключ"_sv, 1, 7 };

```

Для некоторых команд реализованы специальные типы возвращаемых значений. Например, команда [Eval](#) возвращает Variant или BinVariant, что позволяет возвращать произвольные структуры данных из скриптов Lua.

Исполнение одиночных команд

Посылка команды

```

auto async_operation = client.SendCommand( redis::commands::Get{ L"ключ"_sv }
);

```

В качестве результата возвращается объект типа [AsyncOperation](#), реализующий механизмы ожидания и извлечения результатов исполнения команды.

Ожидание результатов

Интерфейс асинхронных команд реализует функциональность для ожидания как одной, так и нескольких команд, в том числе с ограничением по времени.

```

1 | // дождаться готовности результата без ограничения по времени
2 | async_operation.Wait();
3 | // ожидать готовности результата в течение указанного периода времени
4 | // вернёт false, если истекло время ожидания
5 | if( async_operation.WaitFor( Milliseconds{ 100 } ) )
6 |     ...
7 | // ожидать готовности результата до указанного момента времени
8 | // вернёт false, если истекло время ожидания
9 | if( async_operation.WaitUntil( DateTime::Now() + Milliseconds{ 100 } )
10 | )
11 |     ...

```

```

12 std::vector< std::reference_wrapper< Waitable > > async_operations;
13 async_operations.push_back( async_operation.GetWaitable() );
14
15 // дождаться готовности результата всех команд без ограничения по
    времени
16 WaitForAll( std::begin( async_operations ), std::end( async_operations
    ) );
17 // ожидать готовности результата всех команд в течение указанного
    периода времени
18 // вернёт false, если истекло время ожидания
19 if( WaitForAllFor( std::begin( async_operations ), std::end(
    async_operations ),
20                     Milliseconds{ 100 } ) )
21     ...
22
23 // дождаться готовности результата любой из команд без ограничения по
    времени
24 auto iterator = WaitForAny( std::begin( async_operations ),
25                             std::end( async_operations ) );
26 // ожидать готовности результата любой из команд в течение указанного
    периода времени
27 // вернёт false, если истекло время ожидания
28 if( auto optional_iterator = WaitForAnyFor( std::begin(
    async_operations ),
29                                             std::end( async_operations
30 ),
31                                             Milliseconds{ 100 } ) )
    ...

```

Извлечение результата

Возвращаемое командой значение можно получить, вызвав метод [GetResult](#) у объекта асинхронной операции, при этом вызов заблокируется, если результат ещё не готов.

```
auto value = async_operation.GetResult();
```

Если команда возвращает void, вызвав [GetResult](#), можно проверить её исполнение.

```
async_operation.GetResult();
```

В случае отказа исполнения (сетевых проблем, неправильных аргументов, проблем сервера) будет выброшено исключение [RedisException](#) с соответствующим кодом [RedisErrorTypes](#) и сообщением.

Исполнение статических последовательностей команд

Асинхронный клиент поддерживает посылку последовательностей команд на сервер. Это позволяет оптимизировать общение с сервером за счёт упразднения сетевых задержек между отдельными командами.

Один из механизмов клиента – интерфейс статических последовательностей команд, оптимальный для сценария, в котором последовательность команд известна на момент сборки приложения.

Посылка команд

```

1 auto async_batch_operation = client.SendCommands(
2     redis::commands::Get{ L"ключ"_sv },
3     redis::commands::Set{ L"ключ"_sv, L"значение"_sv },
4     redis::commands::Time{} );

```

Возвращаемый объект типа [AsyncBatchOperation](#) аналогичен по функциональности объекту асинхронной операции одиночной команды. Ожидание результатов полностью повторяет ожидание одиночной команды.

Извлечение результата

Для последовательностей команд доступен результат исполнения и возвращаемое значение каждой команды по вызову [GetResult<N>](#), где N – индекс команды в последовательности:

```

1 OptString value = async_batch_operation.GetResult< 0 >();
2 DateTime time = async_batch_operation.GetResult< 2 >();

```

Атомарное исполнение

Redis поддерживает атомарное исполнение последовательностей команд, называемых Redis-транзакцией. Транзакция гарантирует, что никакой запрос от других клиентов не может вклиниться между командами транзакции, и что либо будут исполнены все команды транзакции, либо ни одной. Подробное описание транзакций можно найти в официальной документации: [Transactions – Redis](#).

Посылаются транзакции так же, как и обычные последовательности:

```

1 auto async_batch_operation = client.SendCommandsAtomic(
2     redis::commands::Get{ L"ключ"_sv },
3     redis::commands::Set{ L"ключ"_sv, L"значение"_sv },
4     redis::commands::Time{} );

```

В результатах транзакции добавляется флаг исполнения всей транзакции под индексом 0, как следствие, индексы результатов отдельных команд смещаются на единицу:

```

1 bool executed = async_batch_operation.GetResult< 0 >();
2 if( executed )
3 {
4     OptString value = async_batch_operation.GetResult< 1 >();
5     DateTime time = async_batch_operation.GetResult< 3 >();
6 }

```

Попытка извлечь результаты команд неисполненной транзакции приведёт к выбросу исключения.

Исполнение динамических последовательностей команд

Вдобавок к статическим клиент поддерживает динамические последовательности, состав которых зависит от контекста времени исполнения.

Посылка команд

```

1 redis::AsyncCommandBatch batch;
2 auto get_async_operation = batch.AddCommand( redis::commands::Get{
3     L"ключ"_sv } );
4 batch.AddCommand( redis::commands::Set{ L"ключ"_sv, L"значение"_sv } );
5 auto time_async_operation = batch.AddCommand( redis::commands::Time{
6     } );
7 auto async_batch_operation = client.SendCommands( std::move( batch ) );

```

В ответ на команду, помещаемую в динамическую последовательность, возвращается соответствующий ей [AsyncOperation](#). А вызов метода [AsyncClient::SendCommands](#) возвращает [AsyncOperation](#) для всей последовательности. Ожидание результатов может производиться через любой из возвращённых объектов.

Извлечение результата

Извлечение результатов отдельных команд последовательности аналогично одиночным командам:

```
1 OptString value = get_async_operation.GetResult();
2 DateTime time = time_async_operation.GetResult();
```

Атомарное исполнение

Посылаются транзакции так же, как и обычные последовательности:

```
1 redis::AsyncCommandBatch batch;
2 auto get_async_operation = batch.AddCommand( redis::commands::Get{
  L"ключ"_sv } );
3 batch.AddCommand( redis::commands::Set{ L"ключ"_sv, L"значение"_sv } );
4 auto time_async_operation = batch.AddCommand( redis::commands::Time{
  } );
5 auto async_batch_operation = client.SendCommandsAtomic( std::move(
  batch ) );
```

Отличие только в возвращаемом значении объекта асинхронной операции для всей последовательности. При атомарном исполнении это булевый флаг, указывающий на успешность исполнения транзакции:

```
1 bool executed = async_batch_operation.GetResult();
2 if( executed )
3 {
4     OptString value = get_async_operation.GetResult();
5     DateTime time = time_async_operation.GetResult();
6 }
```

Как и в случае статических последовательностей, попытка извлечь результаты команд неисполненной транзакции приведёт к выбросу исключения.

Настройки асинхронного клиента

Асинхронный клиент Redis позволяет изменять некоторые аспекты своего поведения:

1. Конфиденциальность (Confidential) – позволяет скрывать аргументы команд в логах. По умолчанию аргументы логируются.
2. Кодировка (Encoding) – позволяет устанавливать кодировку, используемую для хранения строк. По умолчанию используется кодировка [CP1251](#).
3. Чтение с мастера (ReadFromMaster) – позволяет перенаправлять команды чтения на мастер кластера Redis. По умолчанию чтение данных выполняется с реплик.
4. Таймаут (Timeout) – позволяет установить предельное время исполнения команды. По умолчанию используется таймаут кластера, если он не выставлен – 30 секунд.
5. Серьёзность ошибок (ErrorSeverity) – позволяет изменять тип сообщений, выводимых в лог при проблемах доступа к БД сервера Redis. По умолчанию фатальными считаются ошибки логики приложения, некорректные параметры подключения, сетевые ошибки и истечение таймаута, остальные логируются как простые ошибки.

Настройки можно выставить при создании клиента, методом [SetSettings](#) или при отправке команды или последовательности команд:

```
1 // Понижаем серьёзность сетевых проблем до предупреждений, а таймаутов
  -- до не критических
2 // ошибок, для всех команд, направляемых через инициализируемый
  экземпляр клиента.
```

```

3  redis::AsyncClient async_client{
4      redis::AsyncClient::FromClusterName,
5      L"Модуль.Хранилище"_s,
6      redis::Settings{
7          .SetErrorSeverity( redis::retNETWORK,
redis::ErrorSeverity::Warning )
8          .SetErrorSeverity( redis::retTIMEOUT, redis::ErrorSeverity::Error
) };
9
10 // Устанавливаем кодировку UTF8 и чтение с мастера для всех последующих
команд с клиента.
11 async_client.SetSettings( redis::Settings{}.SetEncoding( Encoding::UTF8
)
12                                     .SetReadFromMaster( true )
13 );
14 // Отправляем команду Get с таймаутом исполнения в 5 секунд.
15 auto async_operation =
16     async_client.SendCommand( redis::Settings{}.SetTimeout( Seconds{ 5 }
),
17                                     redis::commands::Get{ key } );

```

Настройки, выставляемые при посылке команды, действуют только на передаваемую команду. Во всех вариантах невыставленные настройки остаются при своих текущих значениях.

Особенности работы с асинхронным клиентом Redis



Асинхронный клиент Redis не потокобезопасен!

Обращаться к экземпляру клиента из разных потоков недопустимо. В лучшем случае при одновременном обращении будет выброшено исключение с текстом *"Нельзя посылать новые команды до завершения текущей асинхронной команды!"*, либо процесс сервиса завершится с падением, а в худшем будут испорчены данные на сервере Redis.

Клиенты Redis используют общий пул соединений с кластерами Redis, поэтому кэширование экземпляров клиента Redis в прикладном коде не рекомендуется.

Асинхронный клиент Redis не поддерживает очереди команд, и попытка послать новую команду или последовательность команд до завершения текущей асинхронной операции приведёт к выбросу исключения.

Асинхронный клиент прерывает исполнение команды или последовательности команд с разрывом соединения с сервером в следующих случаях:

1. [AsyncOperation](#) одиночной команды освобождён до завершения исполнения команды.
2. [AsyncBatchOperation](#) статической последовательности команд освобождён до её завершения.
3. Все экземпляры [AsyncOperation](#) динамической последовательности команд освобождены до её завершения.
4. Экземпляр [AsyncClient](#) освобождён до завершения текущей операции.

Расширение асинхронного клиента

Асинхронный клиент поддерживает расширение набора команд в прикладном коде.

Пользовательские команды

В большинстве случаев для добавления команды достаточно унаследовать класс команды от [BaseCommand<T>](#), заменив T на тип возвращаемого командой значения, и передать в конструктор базового класса имя команды и её аргументы.

Например, команду [APPEND](#) можно реализовать следующим образом:


```

1 class Append : public redis::BaseCommand< Int64 >
2 {
3 public:
4     Append( redis::a::Key key, redis::a::Value value )
5         : BaseCommand{ "APPEND"_sv, std::move( key ), std::move( value )
6     }
7     {
8     };

```

Имя команды и имена опций, если таковые имеются, следует передавать как [BinaryStringView](#), а в качестве типов аргументов желательно использовать вспомогательные контейнеры из пространства имён [redis::a](#).

Для реализации более сложных сценариев класс [BaseCommand<T>](#) позволяет предоставлять пользовательские механизмы преобразования команды в формат RESP, форматирования сообщений лога, разбора ответа сервера.

Пользовательские типы аргументов

В качестве аргументов команд расширения можно использовать собственные типы. Для этого достаточно реализовать преобразование типа в формат RESP и форматирование сообщения в лог путём специализации шаблона [Serialization<T>](#).

Например, команду [APPEND](#) с пользовательским типом аргумента можно реализовать следующим образом:

```

1 struct CustomType
2 {
3     Int64 mKey;
4     String mValue;
5 };
6
7 namespace redis
8 {
9     template<>
10 class Serialization< CustomType >
11 {
12 public:
13     static void Log( WStringBuilder& out, const Settings& settings,
14                     const CustomType& value )
15     {
16         Serialization< Int64 >::Log( out, settings, value.mKey );
17         Serialization< StringView >::Log( out, settings, StringView{
18             value.mValue } );
19     }
20     static void Write( std::string& out, const Settings& settings,
21                       const CustomType& value )
22     {
23         Serialization< Int64 >::Write( out, settings, value.mKey );
24         Serialization< StringView >::Write( out, settings, StringView{
25             value.mValue } );
26     }
27     static constexpr size_t Size = Serialization< Int64 >::Size +
28                                     Serialization< StringView >::Size;
29 };
30 } // namespace redis
31
32 class Append : public redis::BaseCommand< Int64 >
33 {
34 public:

```

```

33     Append( const CustomType& custom )
34         : BaseCommand{ "APPEND"_sv, custom }
35     {
36     }
37 };

```

При этом можно, как в примере, воспользоваться имеющимися специализациями шаблона [Serialization<T>](#), либо реализовывать собственную специфику сериализации и логирования.

Пользовательские типы возвращаемых значений

В командах расширения можно использовать собственные типы возвращаемых значений. Для этого требуется реализовать преобразование типа из формата RESP путём специализации шаблона [Deserialization<T>](#).

Например, команду [APPEND](#) с пользовательским типом возвращаемого значения можно реализовать следующим образом:

```

1  enum class AppendStatus
2  {
3      ShortString,
4      LongString,
5      Error
6  };
7
8  namespace redis
9  {
10     template<>
11     class Deserialization< AppendStatus >
12     {
13     public:
14         static std::optional< AppendStatus > Read( ProtocolStream& stream )
15         {
16             if( !stream.ResolveCursor() )
17                 return std::nullopt;
18             switch( stream.GetCursor() )
19             {
20             case ProtocolStream::cINTEGER:
21                 return stream.GetInteger() < 10 ? AppendStatus::ShortString
22                                                         : AppendStatus::LongString;
23             default:
24                 stream.EndFrame();
25                 return AppendStatus::Error;
26             }
27         }
28     };
29 } // namespace redis
30
31 class Append : public redis::BaseCommand< AppendStatus >
32 {
33 public:
34     Append( redis::a::Key key, redis::a::Value value )
35         : BaseCommand{ "APPEND"_sv, std::move( key ), std::move( value ) }
36     {
37     }
38 };

```

Разбора формата RESP производит вспомогательный класс [ProtocolStream](#), предоставляя пользователю интерфейс навигации по типам данных RESP2 в потоке ответа сервера. На момент вызова пользовательского десериализатора пакет ответа сервера может быть принят неполностью, поэтому необходимо вызывать метод `ResolveCursor` и при возвращении им `false` приостанавливать разбор с результатом `std::nullopt`. Десериализатор будет вызван повторно при поступлении следующего сегмента данных в поток ответа. При разборе составных ответов Redis частично разобранный ответ следует хранить в экземпляре десериализатора, т.к. поток приёма ответа однопоточный и не сохраняет вычитанные данные между вызовами десериализатора.